“VERIFICA ALGO”

Matheus Lima e Ranier Veras

DCC606 – Análise de Algoritmos

Universidade Federal de Roraima (UFRR)

O PROBLEMA

O algoritmo analisado consiste em quatro laços de repetição aninhados, nos quais a operação principal é executada diversas vezes dependendo do valor de entrada ***n***. O primeiro laço possui um limite fixo de 10.000 iterações, enquanto os demais dependem diretamente de ***n***, o que faz com que o número total de execuções cresça rapidamente.

```
VerificaAlgo (n: int);
i, j, k, l: int;
para l := 1 TO 10.000 faça
      para i := 1 TO n-5 faça
         para j := i+2 TO n/2 faça
            para k := 1 TO n faça
               {Inspecione elemento}
```

O objetivo da análise é determinar quantas vezes essa operação é realizada e entender como o tempo de execução evolui conforme o tamanho da entrada aumenta. Construir a equação de custo, encontrar a complexidade e realizar uma análise do comportamente assintótico.

DECOMPOSIÇÃO

$$T(n) = \sum_{l=1}^{10.000} \sum_{i=1}^{n-5} \sum_{j=i+2}^{n/2} \sum_{k=1}^{n} 1$$

Simplificação Interna:

$$T(n) = \sum_{l=1}^{10.000} (n^2 - 5n)$$

- A análise detalhada do algoritmo "verificaAlgo" é descontrúida em somatórios aninhados;
- Cada nível de laço representa uma contagem de passos específica em função de n:
  - Laço Externo (l) : Repetição constante (10.000 iterações).
  - Laços Internos (i, j, k): Estrutura de busca e inspeção com dependência linear de n.
- Equação Final:

$$T(n) = \begin{cases} 0, & \text{se } n < 6 \\ 10.000n^2 - 50.000n, & \text{se } n \geq 6 \end{cases}$$

- Conclusão Teórica: Complexidade Assintótica **O(n^2)**.

CONFIGURAÇÕES

Hardware: Execução em memória RAM via alocação dinâmica.

Dados: Entradas de n = {100, 300, 500, 700, 1000, 1500, 2000} em formatos aleatórios, ordenados ascendentes e ordenados descendente.

Rigor: 13 execuções por cenário para reduzir ruídos e garantir estabilidade nas médias.

Medição: Uso de *clock_gettime* com CLOCK_MONOTONIC para precisão em nanossegundos.

Hipótese de Estudo

Validar se o comportamento assintótico teórico ($O(n^2)$) reflete fielmente o tempo de execução em hardware real.

- Usar **Escala Log-Log** para identificar o exponente real de crescimento e observar possíveis desvios causados pela arquitetura do computador.

BUSCA LINEAR

É um algoritmo simples que encontra um elemento em uma lista verificando item por item, do início ao fim, até encontrar o alvo ou percorrer toda a estrutura. É basicamente uma varredura exaustiva de comparação com o valor alvo.

Complexidade: O(n)
Função custo:

$$T(n) = \sum_{i=0}^{n-1} 1$$

Que se resume a: T(n) = n

A Busca Linear é ótima porque não possui redundância; ela acessa cada dado o mínimo de vezes necessário para resolver o problema.

IMPLEMENTAÇÃO

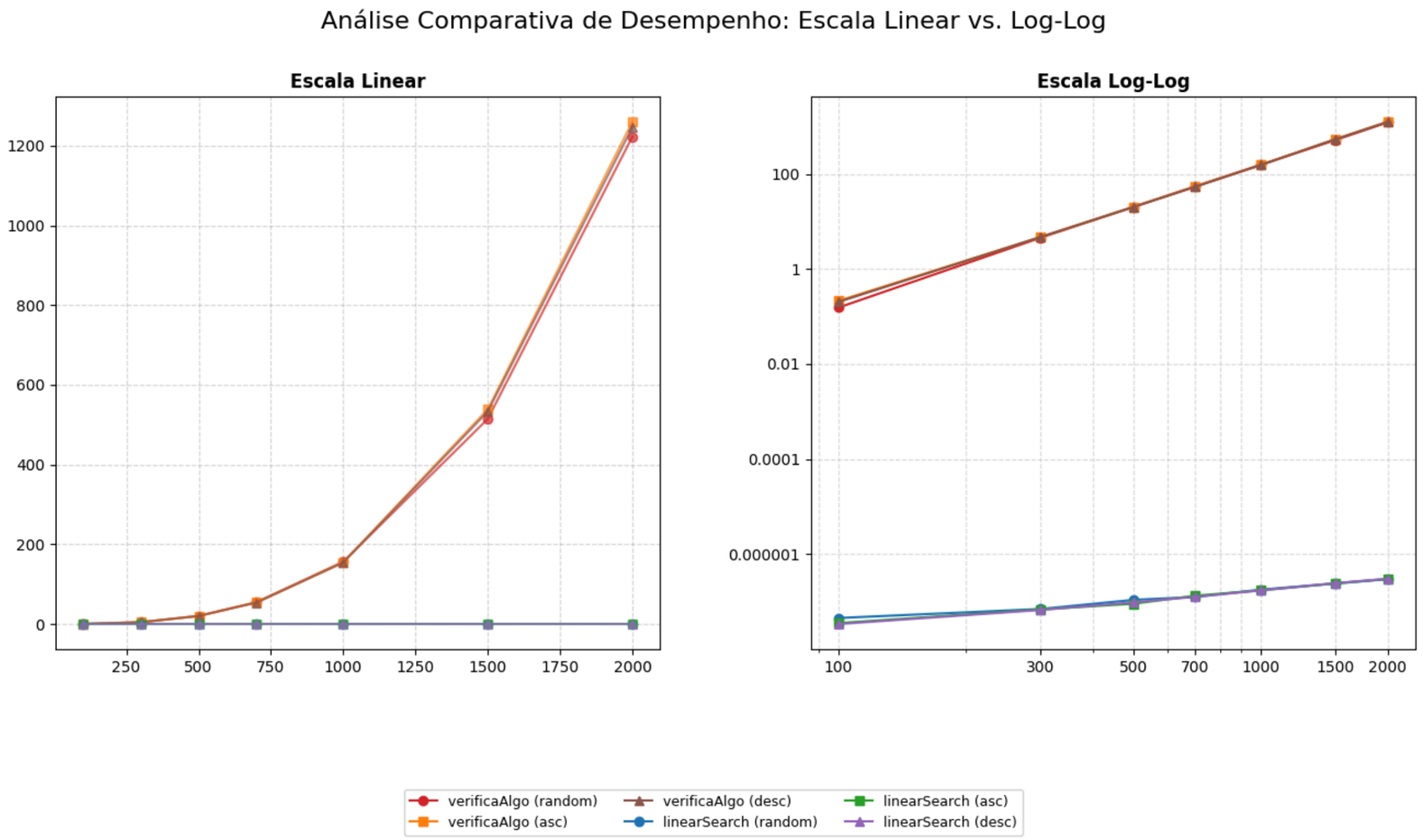
```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>

typedef struct {
    double time;
    unsigned long long inspections;
    size_t memory_bytes;
    int n;
} Metrics;

unsigned long long verificaAlgo(double target, double list[], int list_size) {
    unsigned long long count = 0;
    for (int l = 0; l < 10000; l++) {
        for (int i = 0; i <= list_size - 6; i++) {
            for (int j = i + 2; j <= list_size / 2; j++) {
                for (int k = 0; k < list_size; k++) {
                    if (list[k] == target) {
                        count++;
                    }
                }
            }
        }
    }
    return count;
}
```

```c
unsigned long long linearSearch(double target, double list[], int list_size) {
    unsigned long long count = 0;
    for (int i = 0; i < list_size; i++) {
        if (list[i] == target) {
            count++;
        }
    }
    return count;
}

Metrics runExperiment(unsigned long long (*search_func)(double, double[], int), double target, double list[], int n) {
    struct timespec start, end;
    Metrics m;

    m.n = n;
    m.memory_bytes = n * sizeof(double);

    clock_gettime(CLOCK_MONOTONIC, &start);
    m.inspections = search_func(target, list, n);
    clock_gettime(CLOCK_MONOTONIC, &end);

    m.time = (end.tv_sec - start.tv_sec) +
             (end.tv_nsec - start.tv_nsec) / 1e9;

    return m;
}
```

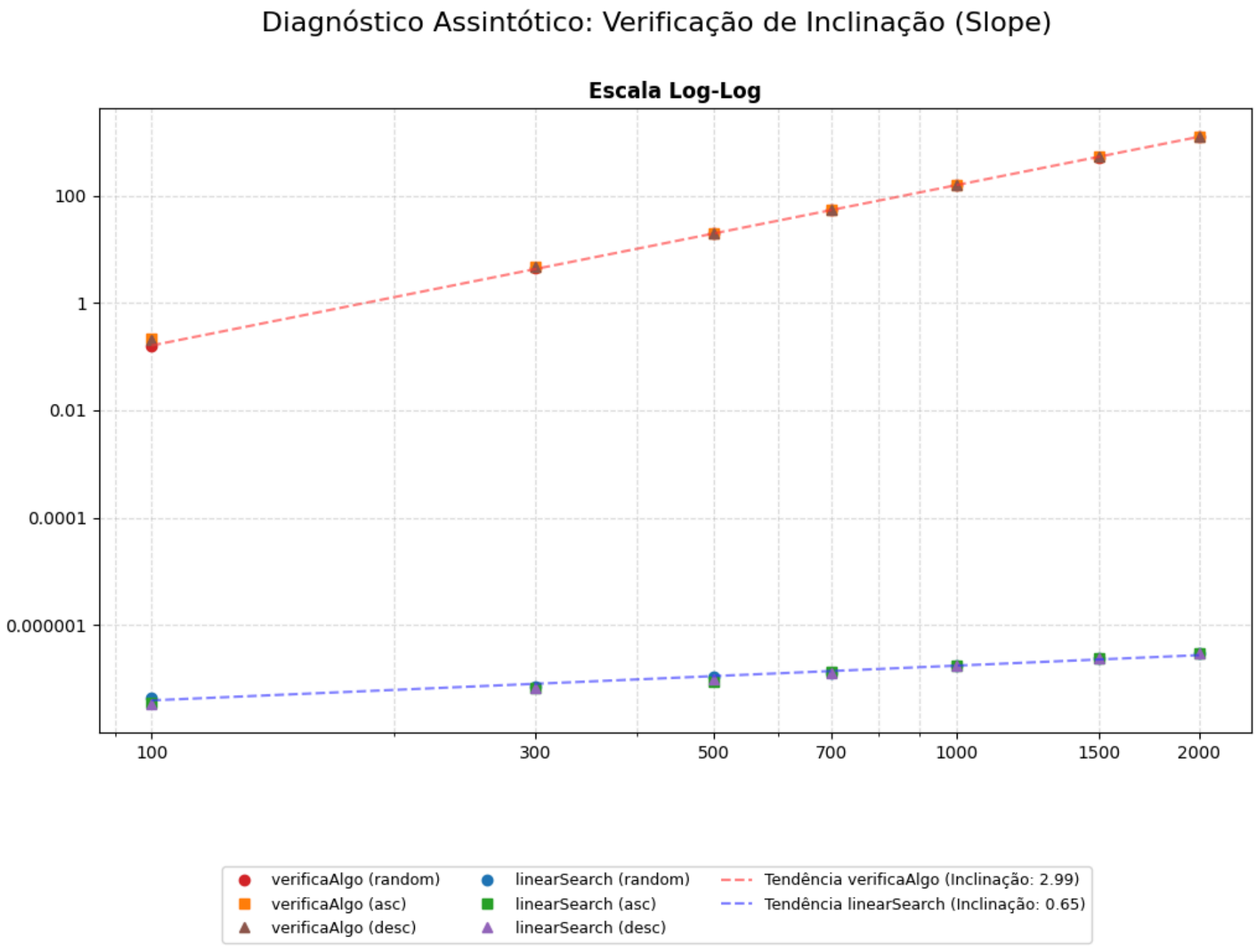
IMPLEMENTAÇÃO

```c
int main() {
    srand(time(NULL));

    FILE *csv_file = fopen("experiment_results.csv", "a");
    if (!csv_file) return 1;

    fprintf(csv_file, "algorithm,data_type,n,run_id,time_sec,inspections\n");

    int sizes[] = {100, 300, 500, 700, 1000, 1500, 2000};
    int num_sizes = sizeof(sizes) / sizeof(sizes[0]);
    char *data_types[] = {"random", "asc", "desc"};
    int num_types = 3;
    int iterations = 13;

    for (int t = 0; t < num_types; t++) {
        for (int s = 0; s < num_sizes; s++) {
            int n = sizes[s];
            char filename[128];

            if (strcmp(data_types[t], "random") == 0)
                snprintf(filename, sizeof(filename), "random_data_%d.bin", n);
            else
                snprintf(filename, sizeof(filename), "sorted_%s_%d.bin", data_types[t], n);

            FILE *bin_fp = fopen(filename, "rb");
            if (!bin_fp) {
                printf("Warning: Could not open %s, skipping...\n", filename);
                continue;
            }
            double *buffer = malloc(n * sizeof(double));
            if (fread(buffer, sizeof(double), n, bin_fp) != (size_t)n) {
                fprintf(stderr, "Error: Failed to read expected data from %s\n", filename);
                free(buffer);
                fclose(bin_fp);
                continue;
            }
            fclose(bin_fp);

            printf("Running experiment: %s (n=%d)\n", filename, n);

            for (int run = 1; run <= iterations; run++) {
                int random_index = rand() % n;
                double target = buffer[random_index];

                Metrics m1 = runExperiment(verificaAlgo, target, buffer, n);
                fprintf(csv_file, "verificaAlgo,%s,%d,%d,%.8f,%llu\n",
                        data_types[t], n, run, m1.time, m1.inspections);

                Metrics m2 = runExperiment(linearSearch, target, buffer, n);
                fprintf(csv_file, "linearSearch,%s,%d,%d,%.8f,%llu\n",
                        data_types[t], n, run, m2.time, m2.inspections);
            }

            free(buffer);
        }
    }

    fclose(csv_file);
    printf("Experiment complete! Results saved to experiment_results.csv\n");
    return 0;
}
```

ANÁLISE


Análise Comparativa de Desempenho: Escala Linear vs. Log-Log
Escala Linear
1200
1000
800
600
400
200
0
250
500
750
1000
1250
1500
1750
2000
Escala Log-Log
100
1
0.01
0.0001
0.000001
100
300
500
700
1000
1500
2000
verificaAlgo (random)
verificaAlgo (asc)
verificaAlgo (desc)
linearSearch (random)
linearSearch (asc)
linearSearch (desc)

ANÁLISE


Diagnóstico Assintótico: Verificação de Inclinação (Slope)
Escala Log-Log
100
1
0.01
0.0001
0.000001
100
300
500
700
1000
1500
2000
verificaAlgo (random)
verificaAlgo (asc)
verificaAlgo (desc)
linearSearch (random)
linearSearch (asc)
linearSearch (desc)
Tendência verificaAlgo (Inclinação: 2.99)
Tendência linearSearch (Inclinação: 0.65)

O PORQUÊ DA DIVERGÊNCIA

A "Constante" Dominante: Para valores n menores que 2000, a constante 10 mil é muito superior ao próprio n.

Estrutura Tripla de Loops: Mesmo que a álgebra simplifique termos, o hardware executa três níveis de laços que dependem de n, são eles: i, j e k. Nesse sentido, matematicamente, o produto de três níveis lineares precisa resutar em uma tendência n^3 no crescimento do tempo.

Convergência Assintótica: O comportamento quadrático (n^2) só poderia ser dominante em valores de n muito altos, onde o peso da constante 10.000 seria desprezível.

Análise Avançada: É possível validar essa tendência através do *Curve Fitting* de Potências (ajuste de curva), que confirma matematicamente o expoente 3 para os dados coletados.

CONCLUSÃO

A teoria $O(n^2)$ descreve o limite ao infinito, mas o experimento revelou que na prática com o intervalo de uso utilizado, essa performance muda, mostrando-se cúbica.

Algoritmos com constantes elevadas e múltiplos laços aninhados podem apresentar comportamentos de ordem superior na prática.

A necessidade de experimentação, da prática, é um fator fundamental na validação, ou contradição, da aplicabilidade de modelos teóricos em sistemas reais.

REFERÊNCIAS

- CORMEN, Thomas H. et al. Introduction to Algorithms. 3. ed. Cambridge: MIT Press, 2009. (Utilizado para fundamentação de somatórios, funções de custo e análise assintótica).
- GEEKSFORGEEKS. Linear Search. Disponível em: https://www.geeksforgeeks.org/linear-search/. Acesso em: 27 abr. 2026. (Utilizado como referência para a implementação e lógica da Busca Linear).

OBRIGADO